Laxmi Manohar Meda

Embedded Software Engineer

(314) 384-9238 • laxmimanoharmeda@gmail.com • https://www.linkedin.com/in/manohar-reddy-246436151/ • Cupertino CA
• https://github.com/yrlmanoharreddy • https://www.linkedin.com/in/manohar-reddy-246436151/

Professional Summary

- Embedded firmware engineer with up to 6 years of experience building production features for consumer devices, focused on low-latency behavior, reliable power and resource management, and robust integration across hardware and software boundaries.
- Design and ship performance-critical firmware in **C** and **C++** for constrained targets, including bounded state machines, peripheral drivers, and recovery paths that align with real product timing and reliability requirements.
- Use **Python** and **Bash** to create reproducible test harnesses, build and verification scripts, and lab automation that keep regression checks consistent across teams and hardware revisions.
- Work with **FreeRTOS**-style patterns and embedded Linux configuration to reason about concurrency, device drivers, and timing guarantees needed for real-time and near–real-time behavior.
- Develop and debug firmware on microcontrollers such as **ARM Cortex-M** and **RISC-V**, interacting directly with peripherals over **SPI**, **I2C**, **UART**, **CAN**, and **USB** while validating protocol framing and throughput on real boards.
- Use scopes, logic analyzers, and multimeters alongside **gdb/JTAG** workflows to isolate signal-integrity issues, race conditions, and fault cases that appear only under hardware stress and corner scenarios.
- Work in teams that use **Git**, CI/CD practices, and **Docker**-based environments to keep builds reproducible, reviews disciplined, and release artifacts traceable from requirement to deployment.
- Collaborate closely with electrical and mechanical engineers to debug physical-digital interactions, apply modular design patterns to firmware components, and mentor junior engineers through practical code reviews and design discussions.

Technical Skills

- **Languages:** C, Embedded C, C++, Python, Java, Bash, Data Structures and Algorithms
- **MCUs & SoCs:** ARM Cortex-M, RISC-V, STM32, NXP Kinetis, ESP32
- **RTOS & Operating Systems:** FreeRTOS, Embedded Linux, Linux kernel configuration, UNIX, MAC, Yocto Project
- **Buses & Protocols:** SPI, I2C, UART, CAN, USB
- **Debug & Instrumentation:** gdb, JTAG, OpenOCD, oscilloscope, logic analyzer, digital multimeter
- **Build & Toolchains:** CMake, Make, GCC, Clang, arm-none-eabi-gcc
- **DevOps & Cloud:** Git, GitHub Actions, GitLab CI, Jenkins, Terraform, Ansible, Docker, Kubernetes, AWS
- **Backend & Frontend:** SpringBoot, FastAPI, RestAPIs, Microservices, databases
- **Generative AI & AI/ML:** LangChain, LangGraph, Retrieval-Augmented Generation (RAG), LLM Orchestration, Agentic Workflows, Prompt Engineering, Embeddings, Hybrid Retrieval,Vector Databases, Hugging Face Transformers, OpenAI, Groq, Gemini, Anthropic, Claude integration, LlamaIndex
- **Testing & QA:** pytest, Unity test framework, Ceedling, Hardware-in-the-loop (HIL) rigs
- **Collaboration:** Jira, Confluence, Github, Bitbucket, Code review tools (Gerrit, GitHub PRs)

Work Experience

Apple Cupertino, CA

Embedded Software / Firmware Engineer Jun 2024 – Present

- Implemented a **C** firmware fallback for a power-gated peripheral path, keeping state transitions bounded while the product stayed inside a rigid boot-time and memory budget.
- Debugged dropped interrupt wakeups in the sensor stack by tightening the handler sequencing and validating the edge cases against timing traces from **JTAG** and the logic analyzer.
- Consolidated duplicate bring-up helpers into a shared **C++** module, trimming ad-hoc logic from the legacy driver layer and making the state machine easier to review safely.
- Patched the USB event loop to process connect, suspend, and resume paths deterministically, leaving the device-side framing logic intact for constrained firmware scheduling.
- Implemented a register-backed transport layer in **C** that handled **SPI**, **I2C**, **UART**, and **USB** transactions with bounded buffering and protocol-framing checks for device stability.
- Validated low-latency peripheral I/O by instrumenting transfer paths with **oscilloscope** captures, then tuned retries and chip-select timing until the bus behavior matched product requirements.

- Built a **Python** regression harness that replayed power-state transitions, watchdog resets, and peripheral faults across 240 scripted scenarios for repeatable firmware bring-up.
- Designed bounded-memory state machines for feature modules that needed safe power-state transition handling, watchdog reset recovery paths, and predictable behavior after brownout conditions.
- Refactored the boot and recovery code so watchdog resets returned through a known initialization path, preserving protocol state cleanly across cold start and warm restart flows.
- Reviewed and merged firmware changes in **Git**, then used **Docker**-based build images and CI/CD jobs to keep release artifacts identical across developer machines and lab rigs.
- Instrumented protocol framing validation on the peripheral interface, checking packet boundaries and CRC behavior against real hardware traces before the feature moved beyond lab validation.
- Maintained a modular driver layout in **C++** for shared buses and feature hooks, keeping ownership clear when multiple teams touched the same hardware path.
- Documented and reproduced signal-integrity faults with a **multimeter** and logic analyzer, then handed calibrated findings to electrical partners during board-level debug sessions.
- Mentored junior engineers through code reviews on interrupt hygiene, buffer ownership, and safe power sequencing, especially where performance pressure tempted shortcuts in the firmware paths.
- Developed internal triage tooling with **LangChain**, **CrewAI**, and a vector database to coordinate multi-step analysis, retrieve prior debug notes quickly, and shorten root-cause searches.

Open Source With SLU Saint Louis, MO

Graduate Research Assistant | Open Source Project Jan 2024 – May 2024

- Pi4Micronaut open source framework for Java-based IoT on Raspberry Pi, some of the core wiring around device access and app side setup
- Micronaut and **Pi4J** integration, so Java components could talk to Raspberry Pi hardware without a lot of extra glue
- Reusable pieces for **GPIO** sensors, actuators, LEDs, switches, displays, motors, and similar devices, mostly common hardware paths
- Java & Micronaut interfaces around hardware functions, kept them reusable.
- Unit and integration tests for component behavior, plus the usual regression fixes when something shifted
- Validation on Raspberry Pi hardware, checking expected behavior with connected devices rather than just mocking everything, Build issues, dependency conflicts, **GPIO** faults, integration bugs, component behavior problems
- Project cleanup across code quality, documentation, testing, build stability, release readiness, not one neat pass, more a set of small fixes
- GitHub flow, branches, pull requests, code reviews, issue tracking, normal open source development work
- Documentation for component setup, hardware interaction, configuration, and usage examples, kept it readable for people wiring things up later

Samsung R&D Institute India Bangalore, India

Embedded Software Engineer May 2020 – Aug 2023

- Stabilized a legacy **C** firmware path by replacing brittle buffer handling with bounded allocations and cleaner module boundaries, removing the flaky retries that had accumulated across earlier releases.
- Refactored **CAN** arbitration handling and serial control paths to stop message collisions under load, then tuned timing checks after **gdb** traces showed a race between interrupt service work and bus access.
- Patched low-level diagnostics in **C** and **Bash** by adding targeted scripts for repetitive lab cycles, while using logic analyzers and oscilloscopes to trace fault windows across validation boards.
- Configured **Embedded Linux** drivers and resource settings, then validated boot-time behavior under real hardware constraints, including interrupt routing, device probing, and memory pressure from concurrent peripherals.
- Optimized firmware allocation patterns to operate within tight memory ceilings on consumer hardware, keeping control paths deterministic while preserving feature coverage for device control and diagnostics.
- Resolved timing drift during cross-team validation by tracing fault states back through **gdb** sessions and serial logs, then coordinating fixes with electrical and mechanical engineers around board-level interactions.
- Documented firmware changes and regression checks in **Git** across iterative builds, supporting 12 hardware validation cycles with traceable diffs, reproducible lab scripts, and clearer handoff notes for integration work.

Education

Saint Louis University Saint Louis, MO

Master's in Computer Science • GPA : 3.98 Jan 2024 – Dec 2024